Purpose

TMP-0 — templates/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/templates/TMP-0_purpose.md`

---
library: templates
id: TMP-0
schema_version: 1.0.0
status: draft

TMP-0 — templates/ Purpose + Boundaries

Purpose
`templates/` defines Axion's **document template library system**:
- what a template is (structure + placeholders + required fields)
- how templates are indexed and versioned
- how templates are selected deterministically from the canonical spec + standards snapshot
- how template rendering is validated for completeness

This library is the source of truth for the **533 TMP templates** and the contracts that govern selection + fill.

What it governs (in scope)
- Template model (template_id, category, version, placeholders, required inputs)
- Template registry/index (what templates exist, where they live, metadata)
- Template selection rules (deterministic mapping from run context → template set)
- Render envelope contract (rendered output + trace metadata + citations)
- Completeness rules (placeholder coverage, required sections present)
- Template deprecation and replacement mapping (recommended)

What it does NOT govern (out of scope)
- Template content correctness for a given domain → comes from canonical/standards/knowledge inputs, not the template system
- Knowledge selection rules → `knowledge/`
- Standards pack content/resolution → `standards/`
- Canonical spec structure → `canonical/`
- Pipeline order/run manifest → `orchestration/`
- Gate DSL semantics → `gates/`
- Risk class meaning/override rules → `policy/`

- Kit packaging rules → `kit/`

Consumers
- Template selection stage (S5)
- Template fill stage (S7)
- Gate evaluation stage (S8) for completeness verification
- Operator UI (template selection justification, missing placeholders)

Determinism requirements
- Given pinned inputs (canonical spec + standards snapshot + template registry + selection rules),
  the selected template set and render results must be reproducible.
- Template IDs are stable; updates require version bumps.
- Selection order is stable and recorded in TEMPLATE_SELECTION artifact.

Piece B — Boundary checklist

File: `Axion/libraries/templates/TMP-0_boundary_checklist.md`

---
library: templates
id: TMP-0a
schema_version: 1.0.0
status: draft

TMP-0a — Boundary Checklist

Belongs in `templates/` if it answers:
- “What templates exist and where are they?”
- “How do we select templates deterministically?”
- “What placeholders/sections are required?”
- “How do we validate template completeness?”

Does NOT belong in `templates/` if it answers:
- “What is the canonical spec schema?” (canonical)
- “What standards apply?” (standards)
- “How do gates evaluate predicates?” (gates)
- “How do we package the kit?” (kit)

Say **“next”** for **TMP-1 Template model (template id, placeholders, rules)**.

Template model

TMP-1 — Template model (template id, placeholders, rules)

Piece A — Template model (concept)

File: `Axion/libraries/templates/TMP-1_template_model.md`

```
---
library: templates
id: TMP-1
schema_version: 1.0.0
status: draft
---

# TMP-1 — Template Model

## Purpose
Define the canonical structure of a template so it can be:
- indexed
- selected deterministically
- rendered with enforceable completeness rules

## What a template is
A template is a document skeleton with:
- stable template_id
- category + subcategory (classification)
- version
- placeholders (what must be filled)
- required inputs (which artifacts it needs)
- output contract (what it produces)
- completeness rules (what "done" means for this template)

## Template types
- doc_template (Markdown/MDX/etc.)
- schema_template (JSON/YAML skeleton)
- checklist_template (structured list)

(You can collapse to one type if you want; the important part is stable metadata + placeholders.)
```

Piece B — Template definition schema (JSON Schema)

File:
`Axion/libraries/templates/schemas/template_definition.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "template_definition.v1",
  "type": "object",
  "required": [
    "template_id",
    "title",
    "category",
    "version",
    "template_type",
    "required_inputs",
    "placeholders",
    "output",
    "created_at",
    "updated_at",
    "owner"
  ],
  "properties": {
    "template_id": { "type": "string", "pattern": "^TMP-[A-Z0-9]{2,3}-\\d{2}$" },
    "title": { "type": "string", "minLength": 3 },

    "category": {
      "type": "string",
      "enum": [
        "PRODUCT_DEFINITION",
        "APPLICATION_BUILD",
        "SYSTEM_ARCHITECTURE",
        "DATA_AND_INFORMATION",
        "EXPERIENCE_DESIGN",
        "INTEGRATIONS_AND_EXTERNAL",
        "SECURITY_PRIVACY_COMPLIANCE",
        "OPERATIONS_AND_RELIABILITY"
      ]
    },

    "subcategory": { "type": "string" },

    "version": { "type": "string", "minLength": 1 },
```

```
    "template_type": { "type": "string", "enum": ["doc_template", "schema_template",
"checklist_template"] },

    "required_inputs": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "minLength": 2 },
      "description": "Contract IDs required to render (CANONICAL_SPEC,
STANDARDS_SNAPSHOT, etc.)"
    },

    "placeholders": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/placeholder" }
    },

    "output": {
      "type": "object",
      "required": ["contract_id", "default_path"],
      "properties": {
        "contract_id": { "type": "string", "minLength": 2 },
        "default_path": { "type": "string", "minLength": 3 },
        "format": { "type": "string", "enum": ["md", "json", "yaml"] }
      },
      "additionalProperties": false
    },

    "completeness": {
      "type": "object",
      "properties": {
        "required_sections": { "type": "array", "items": { "type": "string" } },
        "required_placeholder_ids": { "type": "array", "items": { "type": "string" } }
      },
      "additionalProperties": false
    },

    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" },
    "owner": { "type": "string", "minLength": 2 }
  },

  "$defs": {
    "placeholder": {
```

```
      "type": "object",
      "required": ["placeholder_id", "label", "kind", "required"],
      "properties": {
        "placeholder_id": { "type": "string", "pattern": "^PH-[A-Z0-9_\\-]+$" },
        "label": { "type": "string", "minLength": 2 },

        "kind": { "type": "string", "enum": ["text", "table", "list", "json_block", "code_block"] },
        "required": { "type": "boolean" },

        "source": {
          "type": "object",
          "description": "Where content is derived from (canonical, standards, knowledge, or
computed).",
          "properties": {
            "from": { "type": "string", "enum": ["canonical", "standards", "knowledge", "computed",
"manual"] },
            "pointer": { "type": "string" }
          },
          "additionalProperties": false
        },

        "constraints": {
          "type": "object",
          "properties": {
            "max_words": { "type": "integer", "minimum": 0 },
            "format_hint": { "type": "string" }
          },
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Determinism rules (template identity)

File: `Axion/libraries/templates/TMP-1_determinism_rules.md`

---

library: templates
id: TMP-1a
schema_version: 1.0.0
status: draft

TMP-1a — Determinism Rules (Template Model)

- template_id is immutable; content changes require version bump.
- placeholder_id is immutable within a template version.
- required_inputs must reference known artifact contract IDs (strict mode).
- default_path is stable and used for deterministic kit packaging unless overridden by build plan.

Piece D — Validation checklist

File: `Axion/libraries/templates/TMP-1_validation_checklist.md`

---
library: templates
id: TMP-1b
schema_version: 1.0.0
status: draft

TMP-1b — Validation Checklist

- [ ] template_definition validates against schema
- [ ] template_id matches pattern and is unique in registry
- [ ] placeholder_id values are unique within template
- [ ] required_inputs list is non-empty
- [ ] output.contract_id + default_path present

Template registry/index

TMP-2 — Template registry/index (authoritative lookup for 533 templates)

Piece A — Registry model (concept)

File: `Axion/libraries/templates/TMP-2_registry_model.md`

```
---
library: templates
id: TMP-2
schema_version: 1.0.0
status: draft
---

# TMP-2 — Template Registry / Index

## Purpose
Provide the authoritative registry for:
- what templates exist
- where they live
- their versions and metadata
- whether they're deprecated/replaced
- selection tags (domains, stacks, risk class applicability)

## Design rule
Selection uses the registry, not directory scanning.

## Registry record must include
- template_id
- version
- path
- category/subcategory
- required_inputs
- domains/tags (for selection)
- maturity (draft/reviewed/verified/golden) (optional but recommended)
- deprecated_by (optional)
```

Piece B — Registry entry schema

File: `Axion/libraries/templates/schemas/template_registry_entry.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "template_registry_entry.v1",
  "type": "object",
  "required": ["template_id", "version", "path", "category", "required_inputs", "tags", "maturity"],
  "properties": {
    "template_id": { "type": "string" },
    "version": { "type": "string" },
    "path": { "type": "string" },
    "hash": { "type": "string" },

    "category": { "type": "string" },
    "subcategory": { "type": "string" },

    "required_inputs": { "type": "array", "items": { "type": "string" }, "minItems": 1 },

    "tags": { "type": "array", "items": { "type": "string" } },
    "domains": { "type": "array", "items": { "type": "string" } },
    "profiles": { "type": "array", "items": { "type": "string" } },
    "risk_classes": { "type": "array", "items": { "type": "string" } },

    "maturity": { "type": "string", "enum": ["draft", "reviewed", "verified", "golden"] },

    "deprecated_by": { "type": "string" }
  },
  "additionalProperties": false
}
```

Piece C — Template registry schema (full index)

File: `Axion/libraries/templates/schemas/template_registry.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "template_registry.v1",
  "type": "object",
```

```
  "required": ["registry_id", "schema_version", "templates"],
  "properties": {
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "templates": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "template_registry_entry.v1.schema.json" }
    }
  },
  "additionalProperties": false
}
```

Piece D — Determinism rules (registry behavior)

File: `Axion/libraries/templates/TMP-2_determinism_rules.md`

```
---
library: templates
id: TMP-2a
schema_version: 1.0.0
status: draft
---

# TMP-2a — Determinism Rules (Registry)

- The registry is the source-of-truth; no directory scanning.
- If multiple entries share template_id:
  - choose highest version deterministically (semver, else lexicographic)
  - record chosen version in TEMPLATE_SELECTION
- Template ordering in registry is stable; selection ordering is computed deterministically
(TMP-3).
```

Piece E — Starter registry (example stub)

File: `Axion/libraries/templates/registries/template_registry.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "template_registry.v1",
```

```
  "templates": [
    {
      "template_id": "TMP-01",
      "version": "1.0.0",
      "path": "Axion/libraries/templates/TMP-01.md",
      "category": "PRODUCT_DEFINITION",
      "subcategory": "Problem/Goals",
      "required_inputs": ["CANONICAL_SPEC", "STANDARDS_SNAPSHOT"],
      "tags": ["product", "scope"],
      "domains": ["project"],
      "profiles": ["PROFILE-API", "PROFILE-WEB", "PROFILE-MOBILE"],
      "risk_classes": ["PROTOTYPE", "PROD", "COMPLIANCE"],
      "maturity": "draft"
    }
  ]
}
```

Piece F — Validation checklist

File: `Axion/libraries/templates/TMP-2_validation_checklist.md`

```
---
library: templates
id: TMP-2b
schema_version: 1.0.0
status: draft
---

# TMP-2b — Validation Checklist

- [ ] template_registry validates against schema
- [ ] each template_registry_entry validates
- [ ] template_id uniqueness enforced per chosen version rule
- [ ] paths exist (strict mode) and hashes optional
- [ ] required_inputs list is present and non-empty
```

Template selection rules

TMP-3 — Template selection rules (context → template set)

Piece A — Selection model (concept)

File: `Axion/libraries/templates/TMP-3_selection_model.md`

```
---
library: templates
id: TMP-3
schema_version: 1.0.0
status: draft
---

# TMP-3 — Template Selection Model

## Purpose
Define deterministic rules for selecting an ordered set of templates given:
- canonical spec signals (domains/capabilities/entities)
- standards snapshot (required artifacts, constraints)
- run profile (API/WEB/MOBILE/etc.)
- risk class
- executor type (internal/external) if relevant

## Output artifact
Selection produces TEMPLATE_SELECTION:
- ordered templates
- why each was selected (matching tags/domains/requirements)
- restrictions (risk/maturity constraints)
- any bundles used (optional)

## Design constraints
- Selection is deterministic (same inputs → same ordered list).
- Selection is explainable (each template has a reason).
- Selection is capped (avoid overload).
```

Piece B — Template selection artifact schema

File: `Axion/libraries/templates/schemas/template_selection.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "template_selection.v1",
  "type": "object",
  "required": ["selection_id", "run_id", "selected", "generated_at"],
  "properties": {
    "selection_id": { "type": "string", "pattern": "^TMPSEL-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },
    "generated_at": { "type": "string", "format": "date-time" },

    "selected": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["template_id", "version", "path", "reason", "constraints"],
        "properties": {
          "template_id": { "type": "string" },
          "version": { "type": "string" },
          "path": { "type": "string" },
          "hash": { "type": "string" },

          "reason": {
            "type": "object",
            "required": ["matches"],
            "properties": {
              "matches": { "type": "array", "items": { "type": "string" } },
              "notes": { "type": "array", "items": { "type": "string" } }
            },
            "additionalProperties": false
          },

          "constraints": {
            "type": "object",
            "properties": {
              "required_inputs_present": { "type": "array", "items": { "type": "string" } },
              "risk_class": { "type": "string" }
            },
            "additionalProperties": false
          }
        },
        "additionalProperties": false
      }
```

```
    },

    "cap": {
      "type": "object",
      "properties": {
        "max_templates": { "type": "integer", "minimum": 1 },
        "reason": { "type": "string" }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece C — Selection rules (minimum deterministic algorithm)

File: `Axion/libraries/templates/TMP-3_selection_rules.md`

```
---
library: templates
id: TMP-3a
schema_version: 1.0.0
status: draft
---

# TMP-3a — Selection Rules (Minimum)

## Inputs
- profile_id (from normalized input / run manifest)
- risk_class (from policy)
- canonical domains/capabilities (from canonical_spec)
- standards snapshot required artifacts (optional influence)
- template registry entries

## Filter
A template is eligible if:
1) profile_id is in entry.profiles (or entry.profiles empty)
2) risk_class is in entry.risk_classes (or entry.risk_classes empty)
3) entry is not deprecated (unless repro mode)
4) required_inputs are satisfiable (CANONICAL_SPEC, STANDARDS_SNAPSHOT exist)

## Rank (stable scoring)
```

Score each eligible template:
- +3 if any entry.domains match canonical domains
- +2 if any entry.tags match canonical tags/capabilities
- +1 if category required by standards snapshot (optional rule)
Tie-breakers:
1) category order (fixed)
2) template_id lexicographic
3) version (highest)

Cap selection
- default max_templates = 40 (configurable per profile/risk)
- if over cap, take highest scores deterministically and record cap.reason

---

Piece D — Category ordering registry (fixed order for tie-breaking)

File: `Axion/libraries/templates/registries/template_category_order.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "template_category_order.v1",
  "order": [
    "PRODUCT_DEFINITION",
    "SYSTEM_ARCHITECTURE",
    "APPLICATION_BUILD",
    "DATA_AND_INFORMATION",
    "EXPERIENCE_DESIGN",
    "INTEGRATIONS_AND_EXTERNAL",
    "SECURITY_PRIVACY_COMPLIANCE",
    "OPERATIONS_AND_RELIABILITY"
  ]
}
```

Piece E — Determinism rules

File: `Axion/libraries/templates/TMP-3_determinism_rules.md`

---
library: templates
id: TMP-3b

schema_version: 1.0.0
status: draft

TMP-3b — Determinism Rules (Selection)

- Registry is pinned (template_registry.v1).
- Category order is pinned.
- Scoring and tie-breakers are fixed and recorded.
- Selected list is ordered deterministically and includes reasons.

Piece F — Validation checklist

File: `Axion/libraries/templates/TMP-3_validation_checklist.md`

---
library: templates
id: TMP-3c
schema_version: 1.0.0
status: draft

TMP-3c — Validation Checklist

- [ ] template_selection validates against schema
- [ ] each selected entry includes template_id/version/path/reason
- [ ] selection algorithm is deterministic (same inputs → same ordered output)
- [ ] cap recorded when truncation occurs
- [ ] category ordering is applied consistently

Render envelope schema

TMP-4 — Render envelope schema (output + citations + trace)

Piece A — Render envelope model (concept)

File: `Axion/libraries/templates/TMP-4_render_envelope_model.md`

```
---
library: templates
id: TMP-4
schema_version: 1.0.0
status: draft
---

# TMP-4 — Render Envelope Model

## Purpose
Every rendered template output should have a companion envelope that records:
- what template produced it
- what inputs were used (pointers/refs)
- what knowledge items were cited
- what constraints/policies were applied (use_policy, excerpt limits)
- what verification evidence applies (optional)
This makes renders auditable, replayable, and safe.

## Key idea
Output file contains the content.
Envelope contains metadata + traceability.

## Envelope requirements (minimum)
- envelope_id, run_id
- template_id + version + path
- output_ref (path/hash)
- input_refs (canonical/standards/knowledge selection refs)
- knowledge_citations (KID IDs)
- policy_refs (risk class, policy set)
- created_at
```

Piece B — Render envelope schema (JSON Schema)

File: `Axion/libraries/templates/schemas/render_envelope.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "render_envelope.v1",
  "type": "object",
  "required": [
    "envelope_id",
    "run_id",
    "template_ref",
    "output_ref",
    "input_refs",
    "knowledge_citations",
    "policy_refs",
    "created_at"
  ],
  "properties": {
    "envelope_id": { "type": "string", "pattern": "^ENV-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },

    "template_ref": {
      "type": "object",
      "required": ["template_id", "version", "path"],
      "properties": {
        "template_id": { "type": "string" },
        "version": { "type": "string" },
        "path": { "type": "string" },
        "hash": { "type": "string" }
      },
      "additionalProperties": false
    },

    "output_ref": {
      "type": "object",
      "required": ["path"],
      "properties": {
        "path": { "type": "string" },
        "hash": { "type": "string" }
      },
      "additionalProperties": false
    },

    "input_refs": {
```

```
  "type": "object",
  "required": ["canonical_spec_ref", "standards_snapshot_ref"],
  "properties": {
    "canonical_spec_ref": { "$ref": "#/$defs/ref" },
    "standards_snapshot_ref": { "$ref": "#/$defs/ref" },
    "knowledge_selection_ref": { "$ref": "#/$defs/ref" },
    "plan_refs": {
      "type": "array",
      "items": { "$ref": "#/$defs/ref" }
    }
  },
  "additionalProperties": false
},

"knowledge_citations": {
  "type": "array",
  "items": { "type": "string", "pattern": "^KID-" }
},

"reuse_log_refs": {
  "type": "array",
  "description": "Present only if reusable excerpts were included per allowlist rules.",
  "items": { "$ref": "#/$defs/ref" }
},

"policy_refs": {
  "type": "object",
  "required": ["risk_class", "policy_set_id"],
  "properties": {
    "risk_class": { "type": "string" },
    "policy_set_id": { "type": "string" },
    "decisions": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Policy decision IDs relevant to this render (optional)"
    }
  },
  "additionalProperties": false
},

"completeness": {
  "type": "object",
  "properties": {
    "placeholders_total": { "type": "integer", "minimum": 0 },
```

```
      "placeholders_filled": { "type": "integer", "minimum": 0 },
      "missing_required_placeholder_ids": { "type": "array", "items": { "type": "string" } }
    },
    "additionalProperties": false
  },

  "created_at": { "type": "string", "format": "date-time" }
},

"$defs": {
  "ref": {
    "type": "object",
    "required": ["path"],
    "properties": { "path": { "type": "string" }, "hash": { "type": "string" } },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (envelopes)

File: `Axion/libraries/templates/TMP-4_determinism_rules.md`

```
---
library: templates
id: TMP-4a
schema_version: 1.0.0
status: draft
---

# TMP-4a — Determinism Rules (Render Envelopes)

- Envelope is emitted for every rendered output.
- template_ref points to pinned template version.
- input_refs point to pinned canonical_spec and standards_snapshot (and knowledge_selection if used).
- knowledge_citations list is stable and sorted lexicographically.
- If reuse excerpts are used, reuse_log_refs must be present (and recorded elsewhere).
```

Piece D — Validation checklist

File: `Axion/libraries/templates/TMP-4_validation_checklist.md`

---
library: templates
id: TMP-4b
schema_version: 1.0.0
status: draft

TMP-4b — Validation Checklist

- [ ] render_envelope validates against schema
- [ ] template_ref includes template_id/version/path
- [ ] output_ref.path exists (strict mode)
- [ ] canonical_spec_ref and standards_snapshot_ref present
- [ ] knowledge_citations IDs are valid format and deterministically ordered
- [ ] reuse_log_refs present only when required (allowlisted reuse)

Completeness rules

TMP-5 — Completeness rules (placeholder coverage + required sections)

Piece A — Completeness model (concept)

File: `Axion/libraries/templates/TMP-5_completeness_model.md`

```
---
library: templates
id: TMP-5
schema_version: 1.0.0
status: draft
---

# TMP-5 — Template Completeness Model

## Purpose
Define what it means for a rendered template to be “complete” so
G5_TEMPLATE_COMPLETENESS can be enforced.

## Completeness dimensions
1) **Placeholder completeness**
- all required placeholders are filled
- optional placeholders may be left empty but should be flagged

2) **Section completeness**
- required sections exist (headings/blocks)
- required sections are non-empty

3) **Policy constraints**
- if risk class is PROD/COMPLIANCE, completeness thresholds are stricter

## Outputs
Completeness should be recorded in each render envelope:
- placeholders_total
- placeholders_filled
- missing_required_placeholder_ids
```

Piece B — Placeholder syntax rules (enforceable patterns)

File: `Axion/libraries/templates/TMP-5_placeholder_syntax_rules.md`

---
library: templates
id: TMP-5a
schema_version: 1.0.0
status: draft

TMP-5a — Placeholder Syntax Rules

Allowed placeholder forms (minimum)
Placeholders must use a consistent syntax so they can be detected reliably.

Inline placeholder
- `{{PH:PH-<ID>}}`

Block placeholder
- Start: `<!-- PH:PH-<ID> START -->`
- End: `<!-- PH:PH-<ID> END -->`

Filled vs unfilled
- Unfilled placeholders must remain exactly as placeholder syntax.
- Filled placeholders must not contain placeholder markers.
- “TBD” counts as unfilled unless explicitly allowed.

Forbidden
- Nested placeholders
- Dynamic/templating logic in templates
- Arbitrary code execution

Piece C — Completeness policy registry (thresholds by risk class)

File: `Axion/libraries/templates/registries/template_completeness_policy.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "template_completeness_policy.v1",
  "thresholds": {
    "PROTOTYPE": { "required_placeholders_min_pct": 80, "allow_tbd": true },
```

```
    "PROD": { "required_placeholders_min_pct": 95, "allow_tbd": false },
    "COMPLIANCE": { "required_placeholders_min_pct": 100, "allow_tbd": false }
  }
}
```

Piece D — Completeness evaluation rules

File: `Axion/libraries/templates/TMP-5_evaluation_rules.md`

```
---
library: templates
id: TMP-5b
schema_version: 1.0.0
status: draft
---

# TMP-5b — Completeness Evaluation Rules

## Inputs
- template_definition.placeholders + completeness requirements
- rendered output content
- render_envelope.completeness fields
- risk_class (from policy)

## Rules (minimum)
1) Count required placeholders (required == true).
2) Determine filled required placeholders:
   - placeholder marker no longer present AND content non-empty
   - if content is "TBD"/"TODO" and allow_tbd == false → not filled
3) Compute pct = filled_required / total_required * 100
4) Compare pct to threshold required_placeholders_min_pct for risk_class.
5) Missing required sections cause failure regardless of pct (if template defines
required_sections).

## Output
- Update envelope.completeness:
  - placeholders_total
  - placeholders_filled
  - missing_required_placeholder_ids
```

Piece E — Determinism rules

File: `Axion/libraries/templates/TMP-5_determinism_rules.md`

```
---
library: templates
id: TMP-5c
schema_version: 1.0.0
status: draft
---

# TMP-5c — Determinism Rules (Completeness)

- Placeholder detection uses only TMP-5a allowed syntax.
- Completeness computation uses deterministic counts and thresholds.
- Threshold registry is pinned.
- Missing placeholder lists are sorted lexicographically.
```

Piece F — Validation checklist

File: `Axion/libraries/templates/TMP-5_validation_checklist.md`

```
---
library: templates
id: TMP-5d
schema_version: 1.0.0
status: draft
---

# TMP-5d — Validation Checklist

- [ ] Placeholder syntax rules are explicit and enforceable
- [ ] Completeness policy registry exists and is pinned
- [ ] Completeness evaluation rules depend only on template + rendered output + pinned thresholds
- [ ] Missing required placeholder IDs are deterministic and sorted
```

Gates

TMP-6 — Gates (selection valid, completeness pass)

Piece A — Template gates intent

File: `Axion/libraries/templates/TMP-6_template_gates.md`

---
library: templates
id: TMP-6
schema_version: 1.0.0
status: draft

TMP-6 — Template Gates

Purpose
Ensure templates are selected and rendered deterministically and completely.

Gate set (minimum)

TMP-GATE-01 — Template registry pinned + valid
- template_registry is pinned
- registry validates against template_registry.v1

TMP-GATE-02 — Template selection artifact valid
- TEMPLATE_SELECTION exists
- validates against template_selection.v1
- selected templates exist in registry (template_id/version/path)

TMP-GATE-03 — Required inputs satisfiable
For each selected template:
- required_inputs are present in run manifest artifacts (CANONICAL_SPEC, STANDARDS_SNAPSHOT, etc.)

TMP-GATE-04 — Render envelopes produced
- For each selected template output:
  - a render_envelope exists and validates against render_envelope.v1
  - envelope.template_ref matches the selected template version

TMP-GATE-05 — Completeness threshold met (risk-based)
- completeness computed per TMP-5 rules
- required_placeholders_min_pct threshold met per risk_class

- missing required sections/placeholders fail under PROD/COMPLIANCE

TMP-GATE-06 — Knowledge citations emitted when used
- If knowledge_selection_ref present or knowledge was used:
  - envelope.knowledge_citations must include KID IDs
  - if allowlisted reuse occurred → reuse_log_refs present

Pipeline gate mapping
- G4_TEMPLATE_SELECTION corresponds primarily to TMP-GATE-01..03
- G5_TEMPLATE_COMPLETENESS corresponds primarily to TMP-GATE-04..06

Piece B — Gate mapping files (explicit)

File: `Axion/libraries/templates/TMP-6_gate_mapping.md`

---
library: templates
id: TMP-6a
schema_version: 1.0.0
status: draft

TMP-6a — Mapping to Pipeline Gates

G4_TEMPLATE_SELECTION
Passes if:
- TMP-GATE-01 (registry pinned + valid)
- TMP-GATE-02 (selection artifact valid)
- TMP-GATE-03 (required inputs satisfiable)

G5_TEMPLATE_COMPLETENESS
Passes if:
- TMP-GATE-04 (render envelopes produced)
- TMP-GATE-05 (completeness thresholds met)
- TMP-GATE-06 (citations/reuse logs correct)

Piece C — Machine-readable gate spec (optional)

File: `Axion/libraries/templates/TMP-6_template_gates.spec.json`

{
  "schema_version": "1.0.0",
  "contract_id": "TMP-6b",
  "gates": [
    { "gate_id": "TMP-GATE-01", "name": "Template registry pinned + valid", "severity": "hard_stop", "checks": ["pinned(template_registry)", "schema_valid(template_registry.v1)"] },
    { "gate_id": "TMP-GATE-02", "name": "Template selection artifact valid", "severity": "hard_stop", "checks": ["TEMPLATE_SELECTION exists", "schema_valid(template_selection.v1)", "selected templates exist in registry"] },
    { "gate_id": "TMP-GATE-03", "name": "Required inputs satisfiable", "severity": "hard_stop", "checks": ["for each selected template: required_inputs present"] },
    { "gate_id": "TMP-GATE-04", "name": "Render envelopes produced", "severity": "hard_stop", "checks": ["for each output: render_envelope exists and schema_valid(render_envelope.v1)"] },
    { "gate_id": "TMP-GATE-05", "name": "Completeness threshold met", "severity": "policy_controlled", "checks": ["completeness >= threshold for risk_class", "no missing required sections"] },
    { "gate_id": "TMP-GATE-06", "name": "Knowledge citations + reuse logs", "severity": "hard_stop", "checks": ["if knowledge used then citations present", "if reuse then reuse_log_refs present"] }
  ]
}

Piece D — Evidence requirements

File: `Axion/libraries/templates/TMP-6_evidence_requirements.md`

---
library: templates
id: TMP-6c
schema_version: 1.0.0
status: draft

TMP-6c — Evidence Requirements (Template Failures)

On failure, evidence must include:
- template_id + version
- pointer to registry entry
- pointer to TEMPLATE_SELECTION entry + reason
- missing required inputs (contract_ids) if TMP-GATE-03 fails
- pointer to render_envelope and missing fields if TMP-GATE-04 fails
- missing placeholder IDs and computed pct if TMP-GATE-05 fails

- missing KID citations / reuse logs if TMP-GATE-06 fails

Piece E — Determinism rules

File: `Axion/libraries/templates/TMP-6_determinism_rules.md`

```
---
library: templates
id: TMP-6d
schema_version: 1.0.0
status: draft
---

# TMP-6d — Determinism Rules (Template Gates)

- Gates run in fixed order TMP-GATE-01..06.
- Selection is deterministic from pinned registry + pinned rules.
- Completeness thresholds are pinned and risk-class dependent.
- Evidence lists are deterministically ordered (template_id, placeholder_id).
```

Piece F — Validation checklist

File: `Axion/libraries/templates/TMP-6_validation_checklist.md`

```
---
library: templates
id: TMP-6e
schema_version: 1.0.0
status: draft
---

# TMP-6e — Validation Checklist

- [ ] Template gates cover registry pinning, selection validity, required inputs, envelopes, completeness, citations
- [ ] Gate mapping to G4/G5 is explicit
- [ ] Evidence pointers + remediation included
- [ ] Deterministic ordering enforced for missing items lists
```

Say **“next”** for **TMP-7 Minimum viable templates set**.

Minimum viable templates

TMP-7 — Minimum viable templates set (runtime-required files + “done”)

Piece A — MV templates inventory

File: `Axion/libraries/templates/TMP-7_minimum_viable_set.md`

---
library: templates
id: TMP-7
schema_version: 1.0.0
status: draft

TMP-7 — Minimum Viable templates/ Set

Goal
Define the smallest templates library set required to select and render templates deterministically with completeness enforcement.

Required runtime files (minimum)
1) Core docs
- TMP-0_purpose.md
- TMP-0_boundary_checklist.md
- TMP-1_template_model.md
- TMP-1_determinism_rules.md
- TMP-1_validation_checklist.md
- TMP-2_registry_model.md
- TMP-2_determinism_rules.md
- TMP-2_validation_checklist.md
- TMP-3_selection_model.md
- TMP-3_selection_rules.md
- TMP-3_determinism_rules.md
- TMP-3_validation_checklist.md
- TMP-4_render_envelope_model.md
- TMP-4_determinism_rules.md
- TMP-4_validation_checklist.md
- TMP-5_completeness_model.md
- TMP-5_placeholder_syntax_rules.md
- TMP-5_evaluation_rules.md
- TMP-5_determinism_rules.md
- TMP-5_validation_checklist.md

- TMP-6_template_gates.md
- TMP-6_gate_mapping.md
- TMP-6_evidence_requirements.md
- TMP-6_determinism_rules.md
- TMP-6_validation_checklist.md
- TMP-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/template_definition.v1.schema.json
- schemas/template_registry_entry.v1.schema.json
- schemas/template_registry.v1.schema.json
- schemas/template_selection.v1.schema.json
- schemas/render_envelope.v1.schema.json

3) Registries (runtime defaults)
- registries/template_registry.v1.json
- registries/template_category_order.v1.json
- registries/template_completeness_policy.v1.json

4) Gate specs (optional)
- TMP-6_template_gates.spec.json

Minimum content requirement
To validate the system end-to-end, the library must include at least:
- 1 template_definition file
- 1 registry entry pointing to it
- 1 selection run that produces TEMPLATE_SELECTION
- 1 rendered output + 1 render envelope

Piece B — Definition of done (templates/)

File: `Axion/libraries/templates/TMP-7_definition_of_done.md`

---
library: templates
id: TMP-7a
schema_version: 1.0.0
status: draft

TMP-7a — templates/ Definition of Done

templates/ is “done” when:

Schemas + registries
- [ ] All template schemas validate (JSON Schema check)
- [ ] template_registry validates and references existing template paths
- [ ] template_category_order and completeness policy registries exist and are pinned

Runtime behavior (contract-level)
- [ ] S5 produces TEMPLATE_SELECTION deterministically with reasons
- [ ] S7 renders outputs for selected templates
- [ ] Each output has a render_envelope with citations and trace refs
- [ ] Completeness metrics are computed deterministically and recorded

Gates
- [ ] TMP-GATE-01..06 implemented and mapped to pipeline gates:
  - G4_TEMPLATE_SELECTION
  - G5_TEMPLATE_COMPLETENESS
- [ ] Failures include actionable evidence (missing inputs/placeholders/citations)

Piece C — Minimal folder tree

File: `Axion/libraries/templates/TMP-7_minimal_tree.txt`

```
Axion/libraries/templates/
  TMP-0_purpose.md
  TMP-0_boundary_checklist.md
  TMP-1_template_model.md
  TMP-1_determinism_rules.md
  TMP-1_validation_checklist.md
  TMP-2_registry_model.md
  TMP-2_determinism_rules.md
  TMP-2_validation_checklist.md
  TMP-3_selection_model.md
  TMP-3_selection_rules.md
  TMP-3_determinism_rules.md
  TMP-3_validation_checklist.md
  TMP-4_render_envelope_model.md
  TMP-4_determinism_rules.md
  TMP-4_validation_checklist.md
  TMP-5_completeness_model.md
  TMP-5_placeholder_syntax_rules.md
  TMP-5_evaluation_rules.md
```

```
TMP-5_determinism_rules.md
TMP-5_validation_checklist.md
TMP-6_template_gates.md
TMP-6_gate_mapping.md
TMP-6_evidence_requirements.md
TMP-6_determinism_rules.md
TMP-6_validation_checklist.md
TMP-7_minimum_viable_set.md
TMP-7_definition_of_done.md
TMP-7_minimal_tree.txt

schemas/
  template_definition.v1.schema.json
  template_registry_entry.v1.schema.json
  template_registry.v1.schema.json
  template_selection.v1.schema.json
  render_envelope.v1.schema.json

registries/
  template_registry.v1.json
  template_category_order.v1.json
  template_completeness_policy.v1.json
```

.